XSHELF Manual (Master)

Last updated: 2026-05-26

Intent

XSHELF is built for one outcome: **predictable LLM-assisted work under load**. That means bounded context, schema-validated structured outputs, strict telemetry, and safe automation boundaries.

If you only remember one thing: XSHELF is a runtime substrate, not a chat transcript. It captures relevant repository output, enforces budgets before that output reaches the model, makes structured results checkable, and records enough telemetry to explain what happened later.

XSHELF is designed to fail differently from loose LLM tooling. When it cannot preserve a deterministic contract, it should stop, show the broken boundary, store replayable evidence, and keep stdout/stderr behavior predictable for automation.

This master manual synthesizes three viewpoints:

- a story-first walkthrough (to build the mental model),
- a field guide (to debug and tune quickly),
- an operator playbook (to validate and execute safely).

Architecture Overview

- Canonical engine: `cxrs` (Rust). Bash is fallback only.
- Canonical entrypoint: `bin/xshelf`. `bin/cx` remains the compatibility alias.
- Capture pipeline applies to *system output* only (internal native reduction, then budgets).
- Structured commands are schema-enforced (JSON-only, validated, replayable).
- Schema failures are quarantined (`.codex/quarantine/`) and logged with `quarantine_id`.
- Run telemetry is append-only JSONL (`.codex/cxlogs/runs.jsonl`).
- Policy engine blocks destructive/out-of-repo writes by default.

Runtime Architecture Notes

- Runtime direction: split command logic into cohesive Rust modules.
- Orchestrator: `src/app/mod.rs` (routing + composition).
- Runtime config is centralized in `src/modules/config.rs` (`AppConfig` loaded once at startup).
- Core extracted command modules are grouped by runtime concern:

```
introspect, runtime_controls, agentcmds, logview,
analytics, diagnostics, routing, prompting,
optimize, doctor, schema_ops, settings_cmds,
structured_cmds, task_cmds
```

- Non-schema LLM commands share one execution path in `agentcmds`:

```
execute_llm_command(..., LlmMode)
```

- Behavioral contract is unchanged: deterministic schemas, quarantine/replay, policy gating, budgeted capture, append-only logs.
- Quality gate per extraction slice: `cargo fmt` + `cargo test` must pass.

Quickstart (Backend-Safe)

```
cd <repo-root>
./bin/xshelf version
./bin/xshelf task check --json
./bin/xshelf core --json
./bin/xshelf diag --json --window 20
```

Common Workflows (Copy/Paste)

```
# Structured outputs:
./bin/xshelf diffsum-staged | jq .
./bin/xshelf commitjson | jq .

# Task graph:
./bin/xshelf task fanout "Objective..." --from staged-diff
./bin/xshelf task run-all --status pending

# Optimization:
./bin/xshelf optimize 200
```

Installation / Prereqs

- Required: `bash`
- Required: `git`
- Required: `jq`
- Required for default backend: `codex` CLI
- Development: Rust toolchain (`cargo`, `rustc`)

```
./bin/xshelf doctor
./bin/xshelf capture-status
```

Optional: `ollama` (local backend).

Troubleshooting (Fast)

- Unexpected fallback: inspect routing, then confirm `execution_path`.

```
./bin/xshelf where <cmd>
./bin/xshelf version
```

- Schema command fails: inspect quarantine, then replay the stored response.

```
./bin/xshelf quarantine list
./bin/xshelf replay <id>
```

- Budget clipping surprises: check budget and trace data; reduce capture scope or adjust budgets intentionally.
- Strict log validation fails: legacy rows may predate the current telemetry contract; new runs should conform.
- Backend confusion: run `./bin/xshelf llm show` and verify the selected model when using ollama.

Contents

1 Start Here: Verify What You're Running

Fast path (30 seconds)

```
cd <repo-root>
./bin/xshelf version
./bin/xshelf diag
./bin/xshelf where version diffsum-staged task optimize
```

If `execution_path` is not Rust, you are on fallback. Expect reduced guarantees.

2 The Pipeline (What XSHELF Does With Your Reality)

2.1 Capture and Budgeting

The core reliability move is this: **system output is constrained before it becomes prompt text**.

```
raw system output
  -> internal native reduction
  -> mandatory budgeting (clip/chunk)
  -> embed into prompt
```

Budgets (defaults):

- `CX_CONTEXT_BUDGET_CHARS=12000`
- `CX_CONTEXT_BUDGET_LINES=300`
- `CX_CONTEXT_CLIP_MODE=smart|head|tail` (default `smart`)
- `CX_CONTEXT_CLIP_FOOTER=1`

Inspect current budgets + last-run clip stats:

```
./bin/xshelf budget
./bin/xshelf trace
```

2.2 Modes

- `lean`: minimal prompts/output
- `deterministic`: strict schema-only behavior where applicable
- `verbose`: additional human detail (where supported)

Schema commands always force deterministic mode by default. Override with `CX_SCHEMA_RELAXED=1`.

3 Structured Outputs (Schemas, Quarantine, Replay)

3.1 Schema Registry

Schemas live under `.codex/schemas/`:

- `commitjson.schema.json`
- `diffsum.schema.json`
- `next.schema.json`
- `fixrun.schema.json`

```
./bin/xshelf schema list
./bin/xshelf schema list --json | jq .
```

3.2 Schema Commands

```
./bin/xshelf commitjson | jq .
./bin/xshelf diffsum-staged | jq .
./bin/xshelf next git status | jq .
```

3.3 Failure Procedure

If validation fails, XSHELF writes the raw response + metadata to quarantine and returns non-zero.

```
./bin/xshelf quarantine list
./bin/xshelf quarantine show <id>
./bin/xshelf replay <id>
```

4 Safety Sandbox (Policy Engine)

The policy engine is the boundary between “suggest” and “execute”.

Blocked by default (minimum):

- `sudo`, `rm -rf`, `curl | bash/sh/zsh`
- `chmod/chown` on system paths
- writes outside repo root

```
./bin/xshelf policy show
./bin/xshelf policy check "rm -rf ."
```

5 Task Graph (Fan-out, Run, Run-all)

5.1 Why tasks

Tasks are the unit of migration to parallel work: small, role-tagged, log-linked, and replayable.

```
./bin/xshelf task fanout "Ship release notes improvements" --from staged-
   diff
./bin/xshelf task list --status pending
```

5.2 Run tasks

```
./bin/xshelf task run task_001 --mode deterministic --backend codex
./bin/xshelf task run-all --status pending
```

6 Telemetry (Contract, Validation, Interpretation)

6.1 Log locations

- runs: `.codex/cxlogs/runs.jsonl`
- schema failures: `.codex/cxlogs/schema_failures.jsonl`

6.2 Validate

```
./bin/xshelf logs validate --fix=false
```

Note: strict validation will flag older historical rows that predate the current contract.

6.3 Interpretation (fast heuristics)

- `effective_input_tokens` high: prompts too large or too variable.
- cache trend down: stabilize prompt templates and capture scope.
- frequent clipping: reduce capture scope or raise budgets intentionally.
- schema failures spike: enforce deterministic mode and reduce context ambiguity.

7 Self-Optimization (Pure Analysis)

```
./bin/xshelf optimize 200
./bin/xshelf optimize 200 --json | jq .
```

8 Operator Procedures (Copy/Paste)

Budget stress test

```
CX_CONTEXT_BUDGET_CHARS=2000 CX_CONTEXT_BUDGET_LINES=40 ./bin/xshelf cxo git
    status
./bin/xshelf budget
./bin/xshelf trace
```

Schema integrity check

```
./bin/xshelf commitjson | jq .
./bin/xshelf diffsum-staged | jq .
```

Task loop

```
./bin/xshelf task fanout "Objective: tighten schema errors" --from log
./bin/xshelf task run-all --status pending
./bin/xshelf optimize 200
```